

DISASSEMBLING THE AUTONOMOUS ENTERPRISE ENGINE

From over-the-counter agentic architectures to a governed corporate intelligence operating system

AUTONOMOUS MODELS · INSTITUTIONAL ARCHITECTURES

“Reasoning loops and connections to external and internal resources make autonomous platforms functionally scalable. They do not make them institutionally scalable. The institution receives a complex, partially observable runtime—a black box increasingly composed of other black boxes—that must be understood and governed before production.”

ALEJANDRO REYNOSO

CAMBRIDGE JUDGE BUSINESS SCHOOL · UNIVERSITY OF CAMBRIDGE · JULY 2026

Abstract

This paper introduces the architecture of directly accessible autonomous artificial-intelligence platforms through a deliberate act of disassembly. Rather than presenting a user manual, it opens the engine and identifies the components that transform a natural-language objective into coordinated work: the foundational model, the agent harness, the orchestrator, specialized subagents, procedural Skills, tools, plug-ins, Model Context Protocol connectors, application programming interfaces, code-execution environments, computer-use mechanisms, context construction, knowledge vaults, permissions, validation and artifact production. The paper characterizes these products as over-the-counter, or OTC, autonomous architectures because a user can acquire advanced agentic capabilities without first building a complete institutional AI infrastructure.

The operating-system analogy is then examined with precision. At the level of an individual task, the analogy is persuasive: the platform schedules reasoning processes, allocates work, mediates access to resources, manages working context and produces artifacts. At the level of the enterprise, however, the analogy is incomplete. A corporate operating system must establish identity, mandate, data boundaries, segregation of duties, institutional memory, verification, resilience, auditability and fiduciary accountability across many people, agents and systems. The accessible product therefore constitutes a powerful agentic runtime, not a complete institutional operating system.

The central institutional challenge is the difference between functional and institutional scalability. Reasoning loops and connections to external sources and internal resources allow an OTC product to perform more varied work across a larger capability graph. This functional scalability does not establish repeatability, reliability, authority or accountability. As orchestrators, subagents, retrieval systems and tools are combined, the institution confronts nested black boxes in which context degradation, hallucination, error propagation and behavioral drift can become harder to detect. The paper therefore develops a governance-first path from experimentation to bounded production. It requires the enterprise to understand the engine, construct governance layers and only then authorize production use. This sequence is non-negotiable.

Keywords: autonomous models; agentic systems; Cowork; Skills; Model Context Protocol; knowledge vaults; operating systems; AI governance; auditability; institutional intelligence.

Learning objectives

- ▶ identify the internal components of an accessible autonomous architecture;
- ▶ explain the observe-plan-act-evaluate loop and the role of orchestration;
- ▶ distinguish agents, Skills, tools, MCP connectors and knowledge repositories;
- ▶ evaluate whether an autonomous workspace constitutes an operating system;
- ▶ distinguish functional scalability from institutional scalability;
- ▶ explain why nested agentic complexity increases degradation, hallucination, error-propagation and drift risk;
- ▶ apply the non-negotiable sequence: understand, govern and only then authorize production.

1. Why the engine must be opened

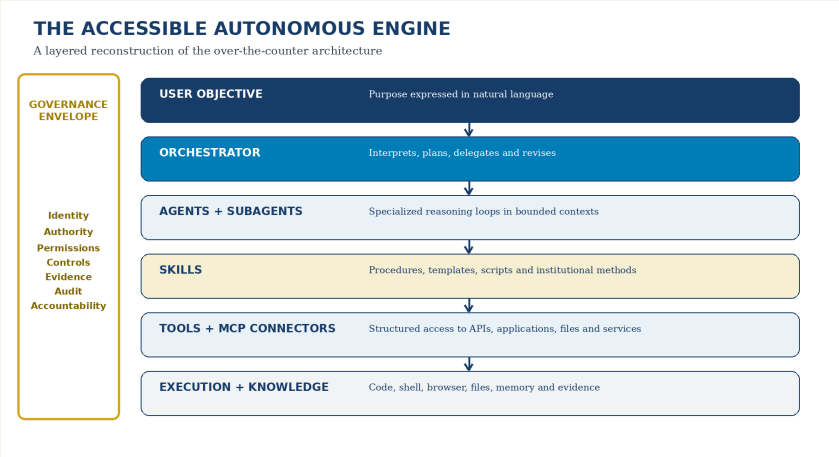
The most visible feature of contemporary artificial intelligence is conversation. The most consequential feature, however, is delegation. A conversational model answers a question. An autonomous system receives an objective and attempts to organize the work necessary to achieve it. This distinction marks the transition from language generation to operational intelligence.

Consider two requests. The first asks the model to explain the financial condition of a company. The second instructs the system to retrieve the latest financial information, reconcile it with the previous quarter, investigate material deviations, construct scenarios, prepare a board presentation and identify decisions requiring immediate attention. The first request is primarily analytical. The second requires an architecture capable of planning, access, execution, verification and artifact production.

PEDAGOGICAL PREMISE

The apparent simplicity belongs to the interface. The object under examination is a complex, partially observable system of reasoning loops, agents, procedures, connectors, tools and knowledge sources. It must be treated as a black box to be opened, not as a simple application to be trusted at face value.

The appropriate method is therefore not to begin with product buttons. It is to disassemble the engine. When the casing is removed, the system reveals a hierarchy: a model capable of reasoning; a harness that maintains state and tool access; an orchestrator that converts objectives into work; specialized agents that operate in bounded contexts; Skills that encode procedures; tools and connectors that reach external systems; and execution environments that turn proposed actions into operational consequences. Each additional layer may conceal its own assumptions, state changes, failure modes and evidence gaps. The result is not one black box but a set of nested black boxes whose interactions must be made governable.



2. What makes an architecture “over the counter”?

The expression over the counter is used by analogy. It describes a packaged environment through which a professional user can obtain sophisticated agentic capabilities without constructing the complete underlying infrastructure. Planning, tool selection, code execution, file manipulation, multi-agent coordination and document production arrive substantially preassembled.

This accessibility is strategically significant. It democratizes experimentation, allows rapid discovery of valuable workflows and makes the future of delegated cognitive work tangible. A user can connect selected resources, state an objective and observe the system move from interpretation to execution. The product becomes a laboratory for redesigning work. Its reasoning loops can be replicated; its tool catalogue can expand; and its access to internal and external resources can deepen. In this limited but important sense, the architecture is functionally scalable.

Characteristic	OTC advantage	Institutional question
Immediate availability	Capability can be tested without a long development cycle.	Who has approved the use case and its risk class?
Natural-language control	Objectives replace long sequences of manual commands.	How are ambiguities converted into bounded specifications?
Preassembled tools	Files, web, code and applications can be reached quickly.	Which permissions are technically available and which are actually authorized?
Flexible general intelligence	The same model can operate across many domains.	Which methods and standards must remain institution-specific?
Rapid artifact production	Reports, code, spreadsheets and presentations emerge quickly.	What evidence proves how the artifact was produced?

The same convenience creates the possibility of premature institutionalization. A platform may perform brilliantly for a knowledgeable individual while remaining unsuitable for consequential corporate use. Ease of access can conceal unresolved questions about identity, data exposure, methodological consistency, tool permissions, knowledge quality, responsibility and release authority.

THE OTC PARADOX

The platform is valuable because it hides complexity and expands functionality. The institution becomes vulnerable when functional scalability is mistaken for institutional scalability, or when hidden complexity is mistaken for controlled simplicity.

3. Agents and the architecture of autonomous loops

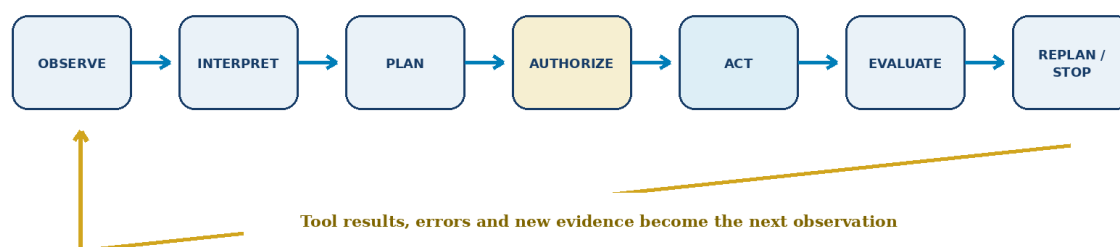
An agent is not simply a language model supplied with a long prompt. A practical agent combines a model with an objective, context, tools, procedural instructions, state, permissions, an execution loop and stopping conditions. The model contributes general reasoning. The harness surrounding it provides continuity and operational boundaries.

WORKING DEFINITION

Agent = model + objective + context + tools + procedures + state + permissions + execution loop.

The fundamental loop is recurrent. The agent observes the request and current state; interprets objectives and constraints; constructs or revises a plan; selects a reasoning step, tool or subagent; checks authority; acts; evaluates the result; and either continues, replans, escalates or stops. Autonomy is therefore not an unstructured freedom. It is a sequence of bounded decisions made under a changing context.

THE AGENTIC LOOP



Autonomy is a controlled recurrence, not a single act of generation.

The loop explains why an autonomous system can recover from incomplete information or failed actions. A tool error is not necessarily terminal. It becomes a new observation. The system may retry, use another source, narrow the task or request human guidance. Reliability depends on whether the harness defines sensible retry limits, error classifications, permission checks and completion criteria.

Recurrence is also a risk multiplier. Every observation, retrieval, summary and tool result can alter the next reasoning step. In longer loops, small errors may be amplified, incomplete context may degrade and an initially plausible plan may drift from the authorized objective. The ability to continue reasoning is therefore not evidence that the reasoning remains reliable.

This architecture also reveals the correct object of audit. The final answer is only the last state of the loop. A reviewer must also inspect material observations, plan revisions, tool calls, exceptions, validation outcomes and human interventions.

4. Orchestrators, subagents and the multiplication of responsibility

Complex assignments are commonly decomposed through an orchestrator-worker pattern. The orchestrator interprets the overall objective, partitions it into work packages, assigns those packages to specialized agents, combines their findings and determines whether the task is complete. Subagents can operate in parallel and in separate contexts, reducing congestion in the principal working memory.

Component	Operational role	Governance question
Orchestrator	Defines the plan, delegates work, integrates results and decides when to stop.	What discretion may it exercise and what decisions remain non-delegable?
Research subagent	Searches internal or external sources for a bounded question.	Which sources and trust levels may it use?
Analytical subagent	Runs calculations, simulations or reconciliations.	Which methods, models and validation tests are approved?
Artifact subagent	Creates documents, spreadsheets, presentations or code.	Which template, release status and provenance must accompany the output?
Validation subagent	Challenges evidence, calculations and completeness.	Is it sufficiently independent from the production path?

Multi-agent design increases capability but also multiplies responsibility. A subagent may receive only a narrow task, yet it can encounter sensitive data or malicious instructions embedded in a source document. Its findings may then be passed to the orchestrator with an undeserved appearance of trust. Inter-agent messages must therefore preserve source identity, original trust level, transformation history, confidence and evidence references.

The architecture now contains black boxes within black boxes: model reasoning inside an agent loop; agent loops inside an orchestration plan; retrieval and tool operations inside each delegated task; and an integration step that may compress uncertainty into a confident final artifact. Complexity can improve task coverage while simultaneously increasing error propagation, hallucination, degradation and drift. The governance burden therefore rises with the depth and connectivity of the agent graph.

CONTROL PRINCIPLE

Delegation should reduce cognitive congestion, not erase provenance. Every subagent conclusion must remain traceable to the evidence and rules that produced it.

At institutional scale, the agent graph itself becomes a governed object. The organization must know which agents may create subagents, which capabilities can be inherited, whether permissions can escalate, how budgets are allocated and which human remains accountable for the combined result.

5. Skills: procedural knowledge between intelligence and action

General intelligence is not the same as institutional competence. A frontier model may understand valuation, credit, law or procurement in broad terms, but professional work depends on approved methods. Skills provide the procedural layer that tells an agent how a category of work should be performed.

A Skill may contain instructions, templates, reference documents, examples, validation criteria, scripts and escalation conditions. The distinction among the core components is concise: the agent decides what must be done; the Skill explains how the organization expects it to be done; the tool supplies the capability with which the action is performed.

Agent	Skill	Tool
Prepare the investment-committee memorandum.	Apply approved valuation methodology, construct three scenarios, disclose conflicts, preserve sources and use the committee template.	Retrieve financial data, execute Python calculations and create the document.
Review a contract.	Follow the institution's clause taxonomy, risk thresholds and escalation rules.	Search the document, compare clauses and write the issue register.
Prepare a client communication.	Use approved tone, disclosure language and release checklist.	Read source files, create a draft and place it in the mail system.

Skills may be preloaded into specialized agents, discovered dynamically from a catalogue, or accessed through delegation to an agent whose role implies a controlled Skill set. Whatever the mapping mechanism, institutional use requires governance comparable to software governance.

SKILL REGISTRY

Every material Skill should have an owner, version, approval date, permitted agents, data classifications, required tools, tests, change history and retirement rule.

Skills are strategic assets because they encode corporate practice. They are also supply-chain risks because they may contain executable scripts. A Skill should therefore be authenticated, version controlled, reviewed for malicious behavior, tested against adversarial inputs and restricted to approved execution environments.

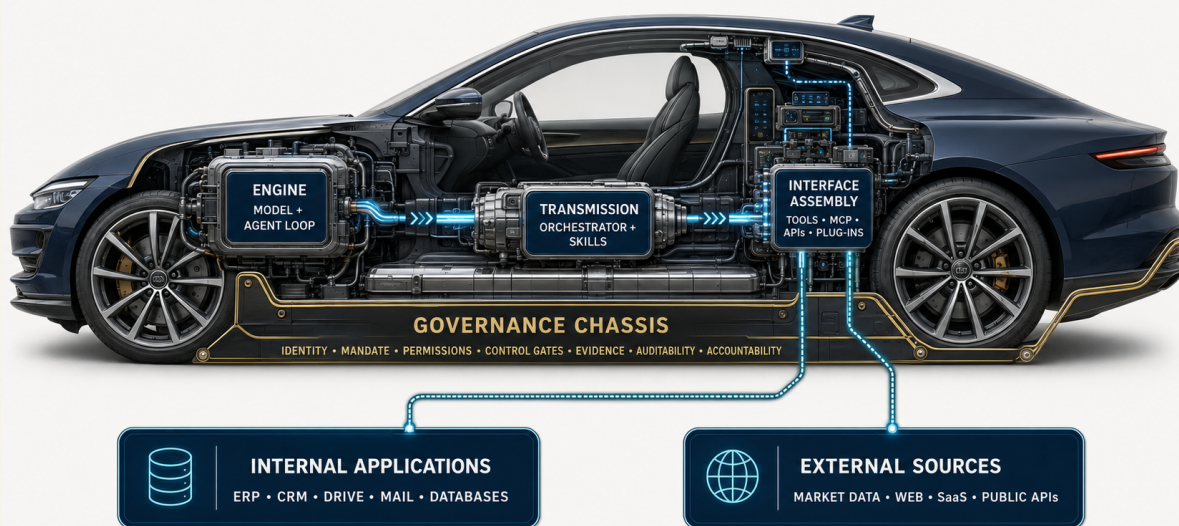
6. Interfaces: plug-ins, MCPs, APIs and connectors

The language model is not normally connected directly to Gmail, Drive, an enterprise resource-planning system or a market-data service. The model operates inside a host or agent runtime. That host constructs context, maintains the loop, registers available tools, checks permissions and executes structured requests through an integration layer. The distinction is indispensable: the model proposes; the surrounding system connects and acts.

This interconnection is the principal source of functional scalability. The same reasoning loop can be extended across additional models, databases, files, applications and services without rebuilding the entire product. Yet every new connection also enlarges the attack surface, context surface, permission graph and number of possible error paths. Connectivity increases capability; it does not supply institutional control.

THE AUTONOMOUS AUTOMOBILE: A PEDAGOGICAL MAP

The model generates power; the surrounding architecture converts it into controlled institutional motion.



The automobile metaphor clarifies the division of labor. The engine creates power, but it does not touch the road directly. A transmission, control system and interface assembly convert power into movement. Likewise, the model produces reasoning, while the host, tools, connectors, MCP servers and APIs convert reasoning into access and action. The governance chassis determines where the system may go, who may drive it and what evidence must remain after the journey.

THE PRECISE CONNECTION

User objective → model reasoning → structured tool request → host permission check → MCP client or native connector → MCP server or adapter → internal or external API → application or data source → structured result → audit record → next agent observation.

6.1 MCP versus plug-ins

MCP and plug-ins are not competing concepts at the same architectural level. A plug-in is a packaged extension installed into a host application. MCP is a standardized communication protocol through which an AI host can discover and invoke external tools, resources and workflows. A plug-in may contain or configure an MCP server, but it may also package agents, Skills, commands, hooks, templates and user-interface elements.

Component	What it is	Automobile analogy
Plug-in	A deployable package that adds a collection of capabilities to the host.	An optional equipment package installed in the vehicle.
MCP	A standard host-client-server protocol for discovering and calling tools and resources.	The standardized socket, wiring and signaling protocol.
Connector	A configured integration with a particular service, including authentication and service-specific settings.	The fitted cable and adapter for a specific destination.
API	The application-specific interface that actually retrieves data or performs an operation.	The mechanical or electronic control exposed by the destination system.
Tool	A bounded operation presented to the agent with a name, description and schema.	A labelled control: search, read, draft, send or execute.
Skill	The procedure governing how the capability should be used.	The driving method, checklist and operating rule.

CORE PRECISION

The plug-in packages capabilities. MCP standardizes communication. The connector configures access. The API performs the service-specific exchange. The tool is the bounded operation visible to the agent. The Skill specifies how the operation should be used.

6.2 How the interface is built

An MCP server is ordinary software implementing the protocol. It declares its identity and capabilities; exposes tools, resources and optional prompt templates; defines structured input schemas; handles authentication and authorization; calls the underlying API, database or local service; normalizes results; and reports errors. The host establishes one or more MCP client sessions and combines discovered capabilities into a tool registry available to the model.

Execution stage	What happens	Control question
1. Connect and initialize	The host establishes a client session with the approved server and negotiates capabilities.	Is this server approved, authenticated and correctly versioned?
2. Discover	The client retrieves available tools, resources and descriptions.	Which capabilities may this agent see?
3. Select	The model proposes a structured invocation based on the objective and tool description.	Is the proposed operation within mandate?
4. Authorize	The host or enterprise gateway checks identity, purpose, scope, parameters and approval rules.	Should this agent, for this user and purpose, be allowed to act now?
5. Execute	The server translates the request into an API call, query or local operation.	Is execution least-privileged and contained?
6. Normalize and return	The result is filtered and returned in structured form.	Are sensitive fields removed and provenance preserved?
7. Continue the loop	The host inserts the result into context; the model evaluates and replans.	Does the next step remain authorized?

For an internal application, the strongest institutional pattern places an enterprise integration gateway between the AI host and corporate systems. The gateway can identify the employee and agent, enforce purpose limitation, apply data classification, separate read from write privileges, require human authorization and create a complete action log. The internal MCP server then wraps one or more corporate APIs and may expose business-semantic tools such as `prepare_credit_snapshot(company_id)`, rather than merely reproducing low-level endpoints.

For an external source, the pattern is similar. A remote connector or MCP server authenticates to the external service—often through OAuth or a service credential—and invokes its API. Email search, Drive retrieval, SaaS operations, public information and market-data access therefore remain mediated exchanges. Credentials should stay in the connector or secure host and should not be placed in the model context.

INTERNAL AND EXTERNAL ACCESS

Internal source: model → governed enterprise gateway → internal MCP server → corporate API → internal application. External source: model → host permission layer → remote connector or MCP server → external API → external service.

A direct API integration is also possible. The host may implement a custom tool function that calls an API without MCP. This can be efficient for a small integration, but it is tightly coupled to the host. MCP adds standardized discovery, schemas and portability across compatible hosts. It should be understood as an interoperability layer, not as a security guarantee.

GOVERNANCE WARNING

MCP answers how an operation can be requested. Governance answers whether this agent, acting for this person, for this purpose, should be permitted to perform it. A standardized interface can still expose an unsafe or malicious capability. Interoperability is functional scalability, not institutional authorization.

7. Code execution, computer use and the transition from answer to action

The ability to write and execute code is one of the principal mechanisms through which a language model becomes operational. An agent can inspect repositories, plan changes, edit files, execute commands, run tests, interpret errors and revise the implementation. The output is no longer a suggestion alone; it can become a changed computational environment.

Execution path	Strength	Risk and control requirement
Structured tool call	Clear semantic meaning and defined input schema.	Validate parameters, scope, identity and result.
Generated orchestration code	Can combine many tool calls and filter intermediate data efficiently.	Inspect transitive action graph, sandbox execution and restrict network access.
Shell or local commands	Highly flexible access to files and software.	Apply command allowlists, path boundaries, resource limits and full logs.
Browser automation	Works where no direct integration exists.	Fragile visual interpretation; require state verification and approval for consequential steps.
Direct computer use	Can operate arbitrary graphical applications.	Highest ambiguity; isolate environment and place irreversible actions behind human gates.

A mature implementation distinguishes execution zones. Remote isolated environments may run code, shell commands and temporary artifacts. The user's computer may expose authorized folders, browser sessions, local applications and local connectors. These zones differ in credential exposure, persistence, network access and forensic visibility. Their boundaries must be documented.

The model itself does not send an email or move money. It proposes an action. A tool, connector or computer-control layer performs the operation. This distinction is essential for accountability because the execution layer can enforce permissions even when the model proposes an impermissible step.

ARCHITECTURAL LESSON

Reasoning authority and execution authority should be separated. The model may recommend; the control layer determines whether the recommendation can become an action.

8. Knowledge vaults and the construction of institutional memory

Autonomous systems become strategically important when connected to the accumulated knowledge of the institution. That knowledge may reside in documents, email, databases, contracts, research repositories, code, board materials, meeting records and prior analyses. No single model context can hold the entire corpus. The architecture must retrieve relevant material just in time.

Knowledge class	Definition	Required treatment
Primary source	Original document, record or dataset.	Preserve unchanged; record authority, date and access path.
Derived material	Summary, classification, extraction or calculation.	Link to sources and transformation logic.
Agent inference	Conclusion produced through model reasoning.	Label explicitly; preserve evidence and confidence.
Human-approved knowledge	Conclusion accepted through accountable review.	Record approver, effective date, scope and review cycle.

An agent can enrich a vault by generating summaries, metadata, links, taxonomies, issue registers, decision records and reusable procedures. Enrichment becomes dangerous when generated conclusions are silently stored beside authoritative sources. An erroneous inference may later be rediscovered and treated as independent evidence, creating a self-reinforcing error loop. When multiple agents retrieve, summarize and reuse one another's outputs, the original error can be obscured by successive transformations and acquire false institutional legitimacy.

EPISTEMIC CONTROL

Source knowledge, derived knowledge, agent inference and human-approved conclusions must remain distinguishable. Institutional memory should be cumulative without becoming credulous.

Every derived knowledge object should therefore carry lineage: source identifiers, creator, model and Skill versions, transformation type, confidence, validation status, approving human, creation date and—where appropriate—an expiration or review date. The knowledge vault is not merely storage. It is an epistemic control system.

9. Does the architecture amount to an operating system?

The operating-system analogy is powerful because the accessible architecture performs functions that resemble process scheduling, resource mediation, memory management and command execution. Agents resemble processes; subagents resemble concurrent workers; Skills resemble libraries and applications; connectors resemble device drivers; context resembles working memory; knowledge repositories resemble a filesystem; and tool traces resemble system logs.

DOES THE OTC ARCHITECTURE AMOUNT TO AN OPERATING SYSTEM?

YES - AS A COGNITIVE RUNTIME

- Receives objectives
- Schedules reasoning processes
- Coordinates agents
- Mediates tool access
- Manages working context
- Executes code and actions
- Produces operational artifacts

NOT YET - AS AN ENTERPRISE OS

- No complete authority model
- No corporate segregation of duties
- No unified knowledge governance
- No enterprise-wide control registry
- No native fiduciary accountability
- No guaranteed audit bundle
- No complete resilience architecture

Conclusion: the OTC product is a powerful agentic runtime. The institution must construct the operating system around it.

At the individual level, the platform can reasonably be described as a cognitive operating environment. It receives objectives, allocates work, mediates resources, maintains state and produces artifacts. Yet an enterprise operating system must coordinate not merely tasks but institutional authority. It must function across legal entities, business units, jurisdictions, information barriers, regulated processes and conflicting mandates.

The product can therefore be functionally scalable while remaining institutionally unscalable. It may connect more models, agents, sources and tools and complete a wider range of tasks. None of those additions proves that the resulting system preserves mandate, quality, security, resilience, evidence or accountability at corporate scale.

The distinction is therefore not semantic. It identifies the missing layers. The OTC product supplies an agentic runtime. The enterprise must supply the constitutional architecture around it: identity, delegated authority, segregation of duties, data classification, approved methodologies, resilience, auditability, incident management and human accountability.

ANSWER TO THE LECTURE QUESTION

Yes, the accessible architecture behaves like an operating system for bounded cognitive work. No, it does not become the company's operating system until the institution embeds its own authority, knowledge and control structure.

10. Why the OTC model encounters a scalability frontier

The central limitation of the OTC solution is not computational capacity or functional extensibility. It is institutional scalability. A workflow may add agents, tools, data sources and reasoning cycles and therefore handle more tasks. It may still fail when repeated across hundreds of employees, many data domains and consequential business processes.

Functional scalability concerns reach: how readily the system can add capabilities and perform more work. Institutional scalability concerns control: whether that expanded system can operate repeatedly within defined authority, reliability, security, evidence and accountability standards. The first is a product property. The second is an institutional achievement. No inference should be made from one to the other.

Scalability frontier	How it appears	Institutional response
Identity and mandate	A user request is treated as implicit authority.	Bind every task to authenticated identity, role and approved purpose.
Permission proliferation	Many agents inherit broad connectors and write capabilities.	Apply least privilege, temporary delegation and capability-specific approvals.
Method inconsistency	Different users create incompatible prompts and Skills.	Maintain an approved Skill registry with version and ownership controls.
Knowledge fragmentation	Useful outputs remain trapped in personal workspaces.	Create governed shared repositories with validation and lineage.
Uncontrolled autonomy	Low-risk automation migrates into consequential action.	Use risk tiers and explicit release or execution gates.
Incomplete traceability	Polished outputs conceal complex execution paths.	Generate an execution-level audit bundle by design.
Vendor dependence	Operations become tied to proprietary interface behavior.	Separate institutional agents, Skills, evidence and policies from vendor runtime.

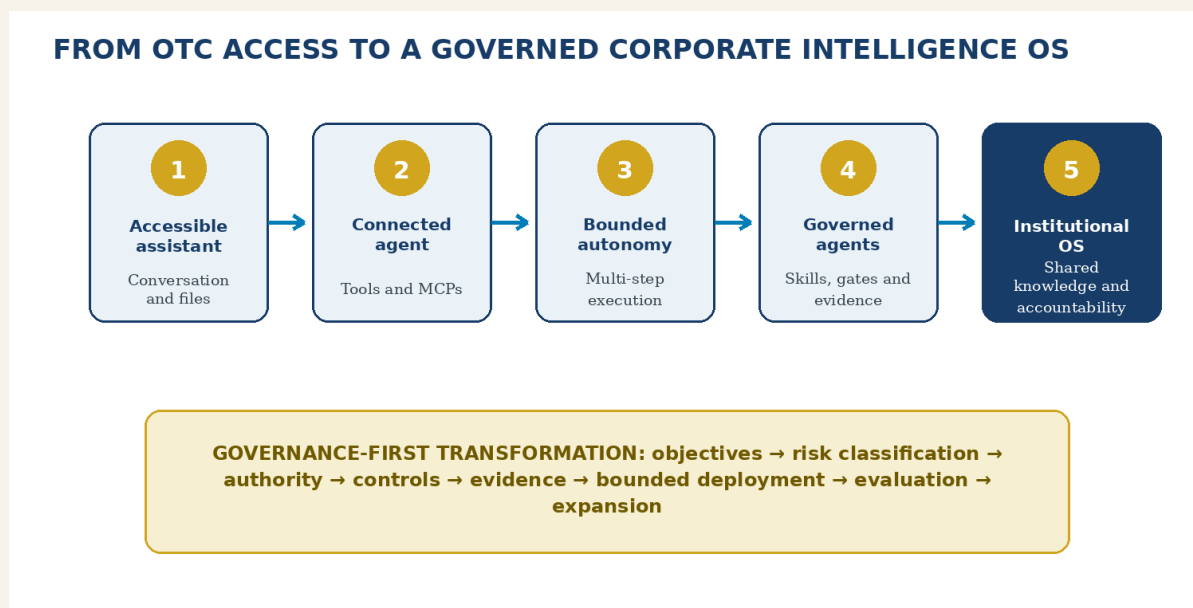
Scalability introduces coordination costs that are invisible in a personal demonstration. The institution must resolve conflicts between agents, manage shared context, prevent cross-domain data leakage, control model and Skill changes, allocate budgets, measure reliability and preserve business continuity. As the architecture becomes more complex, its failure modes also become more compositional: a weak source, misleading retrieval, flawed subagent inference or permissive tool can contaminate downstream reasoning. More complexity means more risk. That risk must be identified, bounded, tested, monitored and managed.

THE SCALING ERROR

Buying additional licenses scales access. Adding loops, agents and connectors scales functionality. Neither action, by itself, scales authority, reliability, consistency, evidence or accountability.

11. Governance first: constructing the institutional layer

A conventional implementation begins with technology, connects data, automates processes and attempts to add controls afterward. Governance-first reverses the sequence. It begins with the company's objectives, classifies the risk of the proposed delegation, assigns authority, defines evidence requirements and only then exposes tools and data. The order is strict: first understand the complete capability and failure graph; second construct and test the governance layers; third authorize bounded production. Production must not be used as the environment in which the institution discovers what it failed to understand or govern.



The correct unit of approval is not the vendor name. It is the capability graph: the specific combination of model, agent, Skill version, tools, data sources, permissions, objective and human controls. The same platform may be acceptable for read-only research and unacceptable for external communication or transaction execution.

Risk tier	Illustrative capability	Minimum governance posture
0 Conversational	No tools and no persistent actions.	Usage policy and ordinary quality review.
1 Read-only analytical	Retrieve and analyze approved information.	Identity, source controls, evidence and validation.
2 Internal artifact creation	Create or modify controlled internal files.	Versioning, release status and change trace.
3 External communication	Send email, publish or communicate with clients.	Named approval, disclosure controls and recipient verification.
4 Transactional	Commit funds, submit filings or alter official records.	Separation of duties, dual authorization and fail-closed execution.
5 Administrative	Change permissions, deploy code or alter infrastructure.	Privileged-access management, independent approval and continuous monitoring.

12. Auditability and the native evidence bundle

A conversational transcript is not an adequate record of autonomous work. The final output must be accompanied by an execution bundle that allows an independent reviewer to reconstruct what happened, test the evidence, challenge the method and verify whether the action was authorized. This requirement becomes more important, not less, as agent graphs become deeper and more interconnected. A polished artifact may otherwise conceal a chain of degraded context, unsupported inference, failed retries and unrecorded transformations.

Audit object	Required content	Question answered
Objective record	Original request, interpreted purpose, material assumptions and acceptance criteria.	What was the system authorized to accomplish?
Agent graph	Orchestrator, subagents, roles and delegated capabilities.	Who or what performed each part of the work?
Context manifest	Policies, instructions, Skills, memories and retrieved sources.	What information and rules shaped the decisions?
Execution trace	Tool calls, code, parameters, results, errors and retries.	What operational path was taken?
Data and method lineage	Source records, transformations, models and formulas.	Where did each material claim come from?
Control record	Permission checks, validation tests, exceptions and approvals.	Which controls operated and what failed?
Artifact manifest	Files, versions, hashes, release status and retention rule.	Is the delivered package complete and intact?

Auditability should be designed at three levels. Outcome audit asks whether the final product is correct. Process audit asks whether approved sources, Skills and tools were used. Control audit asks whether permissions, approvals, containment and escalation rules functioned as intended. An institution needs all three.

NATIVE AUDITABILITY

The audit bundle should be produced as a normal output of the autonomous process, not reconstructed after an incident.

The record should expose operationally relevant reasoning—interpreted objectives, material decisions, evidence, parameters and control outcomes—without requiring disclosure of private chain-of-thought. What matters for governance is reconstructable decision lineage, not an unconstrained stream of internal cognition.

13. From experimentation to governed production

The OTC platform should not be rejected because it is incomplete. It should be used deliberately as the laboratory from which the institution discovers valuable patterns of delegation. The progression begins with individual assistance and may advance through connected and bounded agents. It may enter production only after the relevant capability graph has been understood, governance layers have been implemented and the remaining risk has been accepted by accountable human authority.

The strategic asset is unlikely to be model access alone. Frontier models will be broadly available. Competitive advantage will arise from the architecture constructed around them: proprietary knowledge, carefully engineered Skills, reliable tool integrations, evaluation datasets, decision histories, control systems and the ability to convert each validated process into improved institutional capability.

CORPORATE INTELLIGENCE OPERATING SYSTEM

Frontier models + agentic runtime + institutional agents + governed Skills + controlled connectors + knowledge architecture + identity and permissions + verification + human authority + auditability.

The progression proposed to the board is consequently direct:

1. Understand	2. Govern	3. Production
Expose the complete capability and failure graph: loops, agents, Skills, MCPs, tools, knowledge, execution and control points.	Define and test mandate, registries, permissions, risk tiers, gates, evidence, validation, escalation and accountability.	Authorize only bounded, monitored and reversible use that has satisfied the preceding stages.

The ladder is non-negotiable. Understanding must precede governance; governance must precede production. Skipping the first stage leaves the institution unable to govern the actual system. Skipping the second turns production into uncontrolled experimentation. The objective is not maximum autonomy but bounded production in which authority, evidence, failure boundaries and human accountability are explicit.

For the company, the practical implication is profound. The autonomous architecture should not be treated as another productivity application. It should be treated as a new institutional layer through which objectives, knowledge, procedures, computation and action can be coordinated. Such a layer merits the same seriousness historically devoted to financial controls, information security, operating procedures and delegated authority.

14. Conclusion

When the autonomous engine is opened, no single component explains its power. The model supplies general reasoning. The harness maintains the execution loop. The orchestrator transforms objectives into coordinated work. Subagents provide specialization. Skills encode procedural knowledge. Plug-ins package extensions; MCP standardizes host-to-server communication; connectors configure access; APIs perform service-specific exchanges; and tools expose bounded operations to the agent. Execution environments turn recommendations into actions. Knowledge vaults supply institutional context. Permissions, controls and evidence determine whether the resulting autonomy is legitimate.

Seen from the OTC perspective, these systems already resemble cognitive operating environments. They schedule work, manage context, mediate resources and produce artifacts. Yet the operating-system analogy becomes complete only when the institution adds its own constitutional layer: identity, mandate, segregation of duties, data boundaries, approved methods, verification, resilience, auditability and accountable human judgment. Functional scalability is supplied by loops and connections. Institutional scalability is supplied only by this constitutional layer.

FINAL LESSON

The vendor provides a functionally scalable engine. The institution must understand it, govern it and only then authorize production. It must construct the operating system.

The appropriate sequence is therefore not automation first and governance later. It is: first, understand the engine and its nested failure modes; second, govern the complete capability graph; third, authorize bounded production. The sequence is non-negotiable because complexity compounds risk. More agents, loops and connections may increase functional reach while also increasing degradation, hallucination, error propagation and drift. The ultimate objective is not unrestricted autonomy. It is governed institutional intelligence—an architecture able to reason, coordinate, learn and act while remaining understandable, controllable, auditable and aligned with legitimate corporate purpose.

Discussion questions

- ▶ Which functions of an operating system are already visible in accessible agentic platforms, and which remain institutionally absent?
- ▶ Should Skills be governed as documents, software, models or a distinct category of institutional asset?
- ▶ At what risk tier should an orchestrator be permitted to create subagents dynamically?
- ▶ What evidence would demonstrate that a functionally scalable agent graph is also institutionally scalable?
- ▶ Which controls are required when black-box reasoning loops are nested inside other agents, retrieval systems and execution layers?
- ▶ What evidence would allow a board committee to challenge an autonomous decision without reproducing private chain-of-thought?
- ▶ Which parts of the corporate intelligence operating system should remain vendor-independent?

References

Anthropic (2024). “Building Effective Agents.” Anthropic Engineering.

Anthropic (2026). “Claude Code Plug-ins” and “Model Context Protocol.” Official Claude Documentation.

Model Context Protocol (2025). “Architecture Overview,” “Tools” and “Resources.” Official MCP Documentation and Specification.

National Institute of Standards and Technology (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1.

Raji, I. D. et al. (2020). “Closing the AI Accountability Gap: Defining an End-to-End Framework for Internal Algorithmic Auditing.” Proceedings of the ACM Conference on Fairness, Accountability, and Transparency, 33-44.